

**LAPORAN PRATIKUM
TEKNOLOGI IOT II**

MODUL 4 & 5



Disusun Oleh :

Nama : Sekar Maulia

NPM : 25782112

Kelas : TRI - 3D

Dosen Pengampu :

Dr.Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

**TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG**

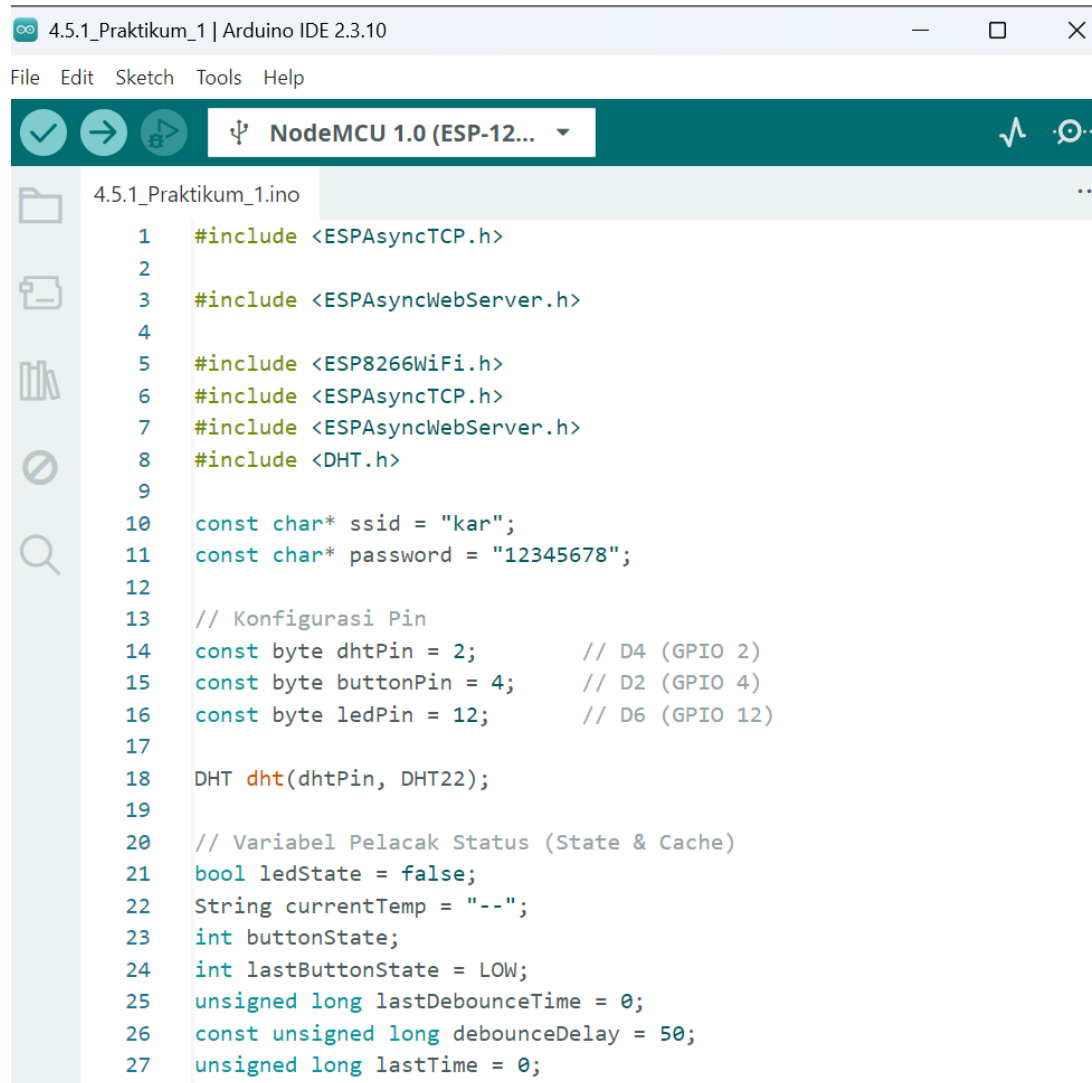
2026/2027

Modul 4: Komunikasi Asinkron dan WebSockets

4.5 Praktikum

4.5.1 Praktikum 1: *Real-Time Dashboard* & Sinkronisasi Tombol Fisik

Kode program



```
4.5.1_Praktikum_1 | Arduino IDE 2.3.10
File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...

4.5.1_Praktikum_1.ino
1  #include <ESPAsyncTCP.h>
2
3  #include <ESPAsyncWebServer.h>
4
5  #include <ESP8266WiFi.h>
6  #include <ESPAsyncTCP.h>
7  #include <ESPAsyncWebServer.h>
8  #include <DHT.h>
9
10 const char* ssid = "kar";
11 const char* password = "12345678";
12
13 // Konfigurasi Pin
14 const byte dhtPin = 2;          // D4 (GPIO 2)
15 const byte buttonPin = 4;       // D2 (GPIO 4)
16 const byte ledPin = 12;         // D6 (GPIO 12)
17
18 DHT dht(dhtPin, DHT22);
19
20 // Variabel Pelacak Status (State & Cache)
21 bool ledState = false;
22 String currentTemp = "--";
23 int buttonState;
24 int lastButtonState = LOW;
25 unsigned long lastDebounceTime = 0;
26 const unsigned long debounceDelay = 50;
27 unsigned long lastTime = 0;
```

4.5.1_Praktikum_1.ino

```

28
29 // Inisialisasi Async Web Server (port 80) & WebSocket (rute /ws)
30 AsyncWebServer server(80);
31 AsyncWebSocket ws("/ws");
32
33 // ----- HTML & JAVASCRIPT (FRONT-END) -----
34 const char index_html[] PROGMEM = R"rawliteral(
35 <!DOCTYPE html>
36 <html>
37 <head>
38   <meta name="viewport" content="width=device-width, initial-scale=1">
39   <title>Real-Time IoT Web</title>
40   <style>
41     body { font-family: Arial; text-align: center; }
42     .card { background: #f0f0f0; margin: 20px auto; padding: 20px; max-width: 300px; border-radius: 10px; }
43     button { padding: 15px 30px; font-size: 20px; border-radius: 5px; cursor: pointer; color: white; }
44     .btn-on { background-color: #4CAF50; }
45     .btn-off { background-color: #f44336; }
46   </style>
47 </head>
48 <body>
49   <h1>Smart Room</h1>
50   <div class="card">
51     <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
52   </div>
53   <div class="card">
54     <h2>LED: <span id="ledStatus">OFF</span></h2>
55     <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn ON</button>
56   </div>

```

4.5.1_Praktikum_1.ino

```

58 <script>
59 // Membuka terowongan koneksi WebSocket ke alamat IP ESP8266
60 var gateway = `ws://${window.location.hostname}/ws`;
61 var websocket;
62
63 // Menjalankan inisiasi saat halaman pertama kali dimuat
64 window.addEventListener('load', onLoad);
65 function onLoad(event) { initWebSocket(); }
66
67 function initWebSocket() {
68   websocket = new WebSocket(gateway);
69   websocket.onopen = onOpen;
70   websocket.onclose = onClose;
71   websocket.onmessage = onMessage;
72 }
73
74 function onOpen(event) { console.log('WebSocket Terkoneksi'); }
75 function onClose(event) { setTimeout(initWebSocket, 2000); }
76
77 // Dipanggil saat tombol di layar web ditekan
78 function toggleLed(){
79   websocket.send('toggle');
80 }
81
82 // Menangkap paket data JSON yang "didorong" (PUSH) oleh C++ NodeMCU
83 function onMessage(event) {
84   var dataObj = JSON.parse(event.data);
85
86   // Menyuntikkan teks angka Suhu ke dalam id "tempValue" HTML

```

4.5.1_Praktikum_1.ino

```

86         // Menyuntikkan teks angka Suhu ke dalam id "tempValue" HTML
87         if(dataObj.suhu != undefined) {
88             document.getElementById('tempValue').innerHTML = dataObj.suhu;
89         }
90
91         // Merombak UI Tombol & Teks sesuai status hardware
92         if(dataObj.led != undefined) {
93             var btn = document.getElementById('toggleBtn');
94             var status = document.getElementById('ledStatus');
95             if(dataObj.led == "1"){
96                 status.innerHTML = "ON";
97                 btn.innerHTML = "Turn OFF";
98                 btn.className = "btn-on";
99             } else {
100                 status.innerHTML = "OFF";
101                 btn.innerHTML = "Turn ON";
102                 btn.className = "btn-off";
103             }
104         }
105     }
106 </script>
107 </body>
108 </html>
109 )rawliteral";

```

4.5.1_Praktikum_1.ino

```

111 // ----- BACK-END & WEBSOCKET LOGIC -----
112
113 // Fungsi menyebarkan paket data JSON ke SELURUH browser pengunjung
114 void notifyClients() {
115     String jsonString = "{\\"led\\":\\" + String(ledState ? 1 : 0) + \\", ";
116     jsonString += "\\"suhu\\":\\" + currentTemp + \\"}";
117     ws.textAll(jsonString);
118 }
119
120 // Handler pesan masuk (dari klik browser)
121 void handleWebSocketMessage(void *arg, uint8_t *data, size_t len) {
122     AwsFrameInfo *info = (AwsFrameInfo*)arg;
123     if (info->final && info->index == 0 && info->len == len && info->opcode == WS_TEXT) {
124         data[len] = 0;
125         if (strcmp((char*)data, "toggle") == 0) {
126             ledState = !ledState;
127             notifyClients();
128         }
129     }
130 }
131
132 // Event handler WebSocket bawaan AsyncWebServer
133 void onEvent(AsyncWebSocket *server, AsyncWebSocketClient *client, AwsEventType type,
134             void *arg, uint8_t *data, size_t len) {
135     switch (type) {
136         case WS_EVT_CONNECT:
137             Serial.printf("Client WebSocket #%u terhubung\\n", client->id());
138             notifyClients(); // Kirim status terkini kepada klien yang baru buka web
139             break;

```

4.5.1_Praktikum_1.ino

```

140     case WS_EVT_DISCONNECT:
141         Serial.printf("Client WebSocket #%u terputus\n", client->id());
142         break;
143     case WS_EVT_DATA:
144         handleWebSocketMessage(arg, data, len);
145         break;
146     }
147 }
148
149 void setup() {
150     Serial.begin(115200);
151
152     // Mengatur pin tombol sebagai input (dengan resistor pull-down eksternal)
153     pinMode(buttonPin, INPUT);
154
155     pinMode(ledPin, OUTPUT);
156     digitalWrite(ledPin, LOW);
157     dht.begin();
158
159     WiFi.mode(WIFI_STA);
160     WiFi.begin(ssid, password);
161     while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
162     Serial.println("\nIP Address: " + WiFi.localIP().toString());
163
164     server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
165         request->send_P(200, "text/html", index_html);
166     });
167

```

4.5.1_Praktikum_1.ino

```

168     // Ikat handler WebSockets ke server utama
169     ws.onEvent(onEvent);
170     server.addHandler(&ws);
171     server.begin();
172 }
173
174 void loop() {
175     ws.cleanupClients();
176
177     // 1. Eksekusi perangkat keras LED
178     digitalWrite(ledPin, ledState ? HIGH : LOW);
179
180     // 2. Baca Tombol Fisik (Debounce) tanpa nge-freeze program
181     int reading = digitalRead(buttonPin);
182     if (reading != lastButtonState) {
183         lastDebounceTime = millis();
184     }
185     if ((millis() - lastDebounceTime) > debounceDelay) {
186         if (reading != buttonState) {
187             buttonState = reading;
188
189             // Karena menggunakan resistor PULL-DOWN eksternal, tombol ditekan = HIGH
190             if (buttonState == HIGH) {
191                 ledState = !ledState;
192                 notifyClients(); // PUSH DATA KE BROWSER SEKETIKA
193             }
194         }
195     }
196     lastButtonState = reading;

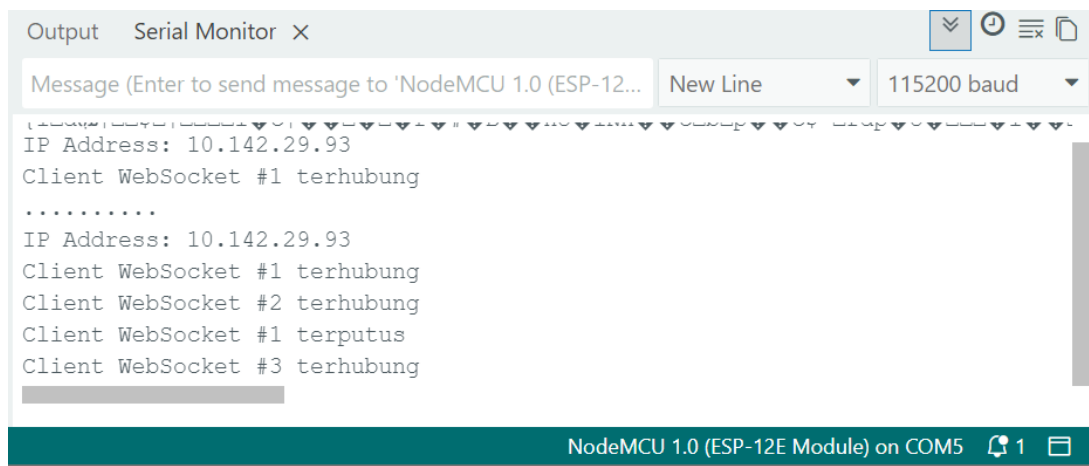
```

```

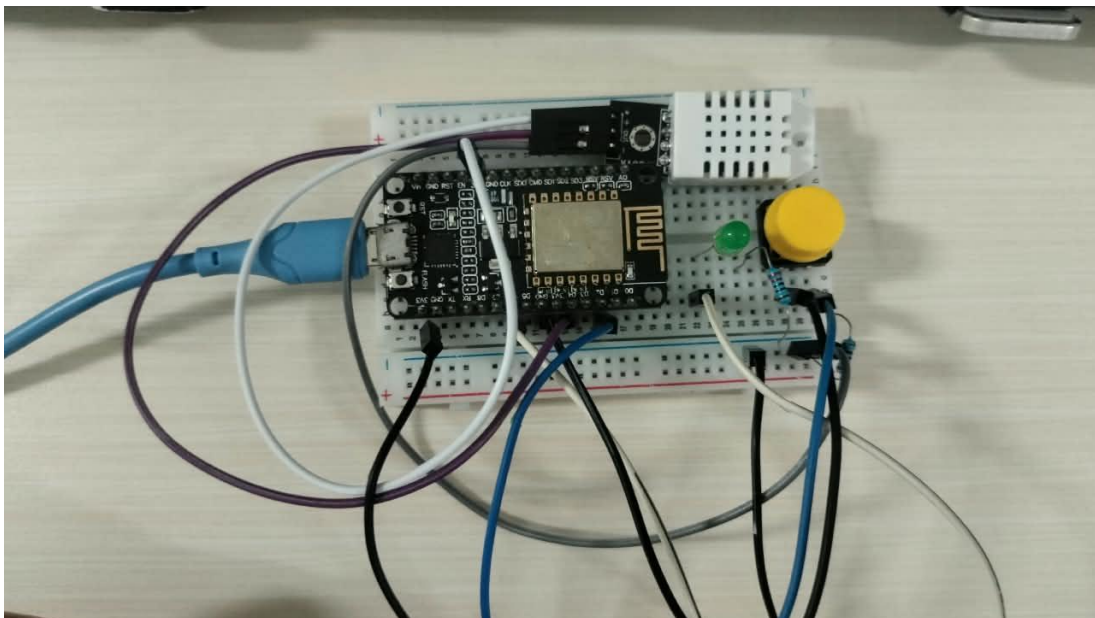
196     lastButtonState = reading;
197
198     // 3. Baca DHT setiap 3 detik di memori cache, tanpa memblokir CPU
199     if ((millis() - lastTime) > 3000) {
200         float t = dht.readTemperature();
201         if(!isnan(t)) {
202             currentTemp = String(t);
203             notifyClients(); // Push pembaruan suhu otomatis
204         }
205         lastTime = millis();
206     }
207 }

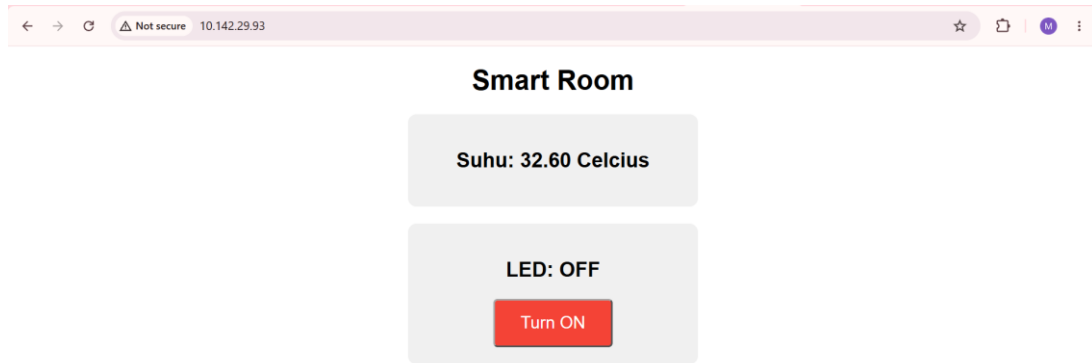
```

Serial monitor

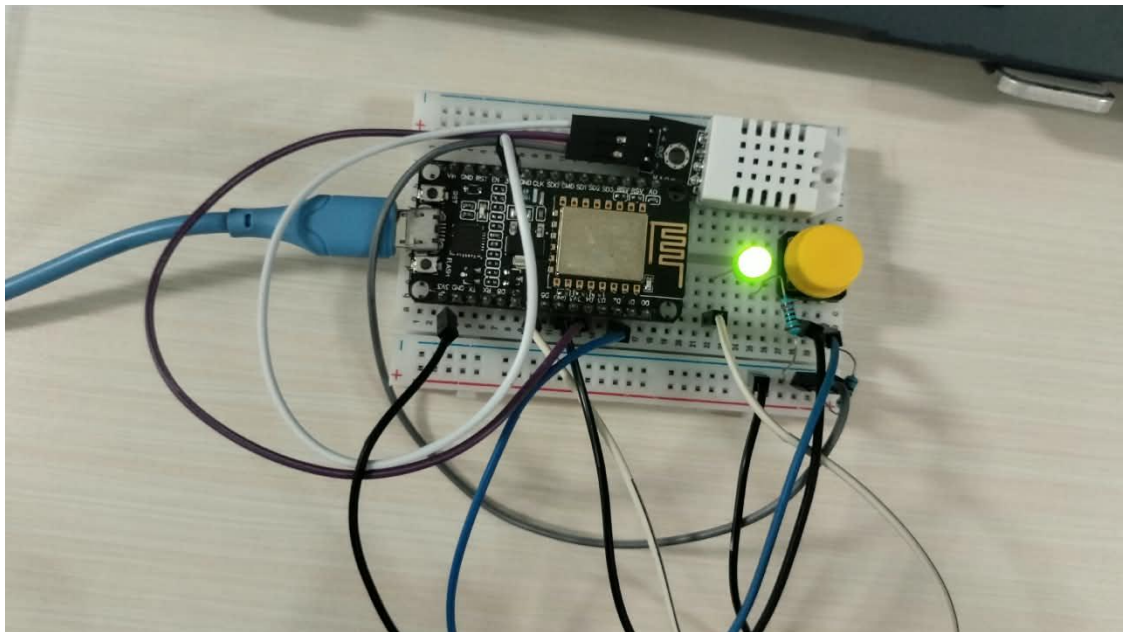


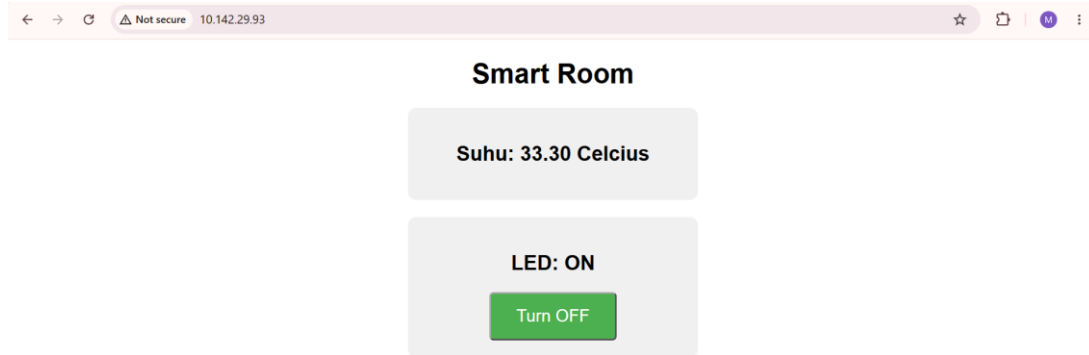
Rangkaian Fisik Awal dan tampilan dashboarnya pertama kali dibuka





Lalu jika kondisi button ditekan ataupun led di tampilan doashbord nya on, maka lampu led akan menyala seperti berikut





4.6 Latihan

- **Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Jaawab:

WebSockets lebih hemat bandwidth karena koneksi antara browser dan NodeMCU tetap terhubung, sehingga data yang dikirim hanya bagian yang mengalami perubahan, seperti nilai suhu atau status LED. Berbeda dengan Meta Refresh HTTP yang harus memuat ulang seluruh halaman web setiap beberapa detik.

Selain itu, WebSockets tidak membuat layar berkedip karena halaman HTML tidak perlu di-refresh secara keseluruhan. Data baru langsung dikirim dari NodeMCU ke browser dan hanya bagian tampilan tertentu yang diperbarui. Karena itu, tampilan menjadi lebih halus, cepat, dan terasa real-time.

- **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

Jawab:

Jika menggunakan `delay(3000)` di dalam `void loop()`, NodeMCU akan berhenti menjalankan proses lain selama 3 detik. Akibatnya, komunikasi WebSockets bisa terlambat merespons karena NodeMCU tidak dapat langsung menerima atau mengirim data selama waktu tersebut.

Hal ini dapat membuat respon tombol pada browser maupun tombol fisik terasa lambat atau lag. Karena itu, pada sistem ini digunakan `millis()` agar pembacaan sensor tetap dilakukan setiap beberapa detik tanpa menghentikan proses komunikasi WebSockets yang harus berjalan secara real-time.

- **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok `// Merombak UI Tombol & Teks sesuai status hardware`. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

Jawab:

ID elemen HTML yang menjadi target pengubahan oleh JavaScript adalah:

1. `tempValue`, digunakan untuk menampilkan dan memperbarui nilai suhu.
2. `ledStatus`, digunakan untuk mengubah teks status LED menjadi ON atau OFF.
3. `toggleBtn`, digunakan untuk mengubah tulisan tombol menjadi Turn ON atau Turn OFF sekaligus mengubah tampilan kelas tombol.

Jadi, saat NodeMCU mengirim data baru melalui WebSockets, JavaScript akan memperbarui bagian-bagian tersebut tanpa perlu me-refresh seluruh halaman web.

- **Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:
 - i. **Sisi Back-End (C++):** Buat variabel penampung (misal: `String currentHum = "--";`). Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer *non-blocking* `millis()`. Perbarui fungsi `notifyClients()` agar menyisipkan data kelembapan ke

dalam paket JSON yang dikirim ke *browser* (contoh format:

```
{"led": "...", "suhu": "...", "hum": "..."}).
```

- ii. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (`<div class="card">`) baru pada kerangka HTML sebagai penampil kelembapan (misal: `<h2>Kelembapan: <span id="humValue">--</span> %</h2>`). Kemudian, modifikasi fungsi JavaScript `onMessage(event)` agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen `humValue`.

Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

Jawab:

- Sisi C++

Tambahkan variabel untuk menyimpan nilai kelembapan:

```
// Variabel Pelacak Status (State & Cache)
bool ledState = false;
String currentTemp = "--";
String currentHum = "--";
```

Kemudian ubah fungsi `notifyClients()` agar data kelembapan ikut dikirim dalam format JSON:

```
// Fungsi menyebarkan paket data JSON ke SELURUH browser pengunjung
void notifyClients() {
    String jsonString = "{\"led\": \"\" + String(ledState ? 1 : 0) + "\", ";
    jsonString += "\"suhu\": \"\" + currentTemp + "\", ";
    jsonString += "\"hum\": \"\" + currentHum + "\"}";
    ws.textAll(jsonString);
}
```

Lalu pada bagian pembacaan sensor DHT, tambahkan pembacaan kelembapan:

```
// 3. Baca DHT setiap 3 detik di memori cache, tanpa memblokir CPU
if ((millis() - lastTime) > 3000) {
    float t = dht.readTemperature();
    float h = dht.readHumidity();

    if(!isnan(t)) {
        currentTemp = String(t);
    }

    if (!isnan(h)) {
        currentHum = String(h);
    }

    notifyClients(); // Push pembaruan suhu otomatis
    lastTime = millis();
}
```

- Bagian HTML

Tambahkan kartu baru untuk menampilkan nilai kelembapan:

```
<div class="card">
  <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
</div>
<div class="card">
  <h2>Kelembapan: <span id="humValue">--</span> %</h2>
</div>
<div class="card">
  <h2>LED: <span id="ledStatus">OFF</span></h2>
  <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn ON</button>
</div>
```

- Bagian JavaScript

Pada fungsi onMessage(event), tambahkan kode berikut:

```
// Menyuntikkan teks angka Suhu ke dalam id "tempValue" HTML
if(dataObj.suhu !== undefined) {
    document.getElementById('tempValue').innerHTML = dataObj.suhu;
}

if(dataObj.hum !== undefined) {
    document.getElementById('humValue').innerHTML = dataObj.hum;
}
```

Serial Monitor



Tampilan dashboard



Smart Room

Suhu: 32.00 Celcius

Kelembapan: 53.50 %

LED: OFF

Turn ON



Smart Room

Suhu: 32.20 Celcius

Kelembapan: 60.30 %

LED: ON

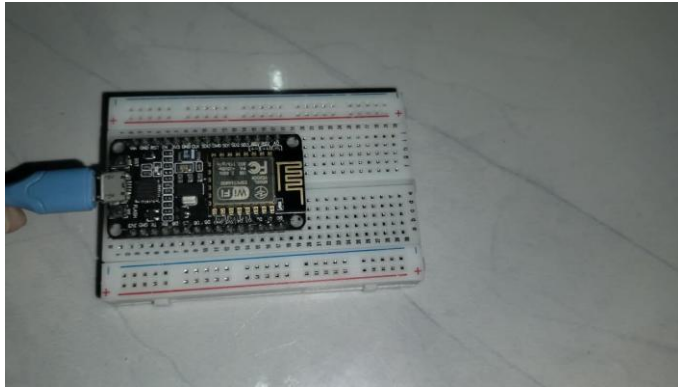
Turn OFF

Modul 5: Topologi Jaringan Terdesentralisasi (ESP-NOW dan Mesh)

5.5 Praktikum

5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU

Perangkat yang digunakan:



Kode program yang digunakan

5.5.1_Praktikum_1.ino

```
1  #include <ESP8266WiFi.h>
2
3  void setup() {
4      Serial.begin(115200);
5      Serial.println();
6
7      // Mengaktifkan mode Station agar MAC address antarmuka STA aktif
8      WiFi.mode(WIFI_STA);
9      WiFi.disconnect();
10
11     Serial.print("ESP8266 MAC Address: ");
12     Serial.println(WiFi.macAddress());
13 }
14
15 void loop() {
16     // Tidak ada proses berulang
17 }
```

Serial Monitor

Output Serial Monitor X

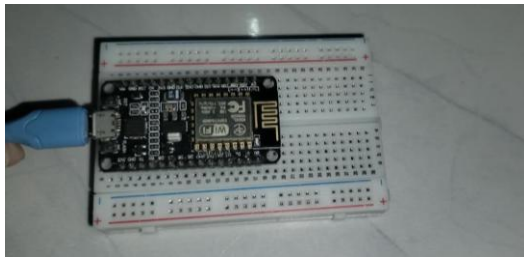
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

17:18:26.466 -> rd\$|\$\$|1b|{p"n\$
17:19:04.177 -> ESP8266 MAC Address: 80:7D:3A:16:C6:4A

Tampilan serial monitor ini menunjukkan bahwa mac address dari esp8266 yang saya gunakan adalah 80:7D:3A:16:C6:4A

5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

Perangkat yang digunakan



Saya disini berperan sebagai controller

Kode praktikum yang diunggah ke nodemcu nya adalah sebagai berikut, karena kelompok saya beranggotakan 4 org, jadi bagian mac receiver nya ditambah satu

Berikut adalah MAC Address perangkat dari masing masing anggota kelompok:

MAC Address Sekar (saya sendiri) : 80:7D:3A:16:C6:4A

MAC Address Crossela : EC:FA:BC:10:62:3C

MAC Address Lilis: E8:DB:84:88:19:9E

MAC Address Mita 2C:F4:32:8E:A3:D7

5.5.2_Praktikum_2.ino

```
1  #include <ESP8266WiFi.h>
2  #include <espnow.h>
3
4  // MAC Address Receiver
5  uint8_t receiver1[] = {0xEC, 0xFA, 0xBC, 0x10, 0x62, 0x3C}; // Sela
6  uint8_t receiver2[] = {0xE8, 0xDB, 0x84, 0x88, 0x19, 0x9E}; // Lilis
7  uint8_t receiver3[] = {0x2C, 0xF4, 0x32, 0x8E, 0xA3, 0xD7}; // Mita
8
9  typedef struct struct_pesan {
10     int perintahId;
11     int nilaiParameter;
12 } struct_pesan;
13
14 struct_pesan paketKirim;
15
16 unsigned long previousMillis = 0;
17 const long interval = 2000;
18
19 void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
20     char macStr[18];
21
22     snprintf(macStr, sizeof(macStr),
23              "%02x:%02x:%02x:%02x:%02x:%02x",
24              mac_addr[0], mac_addr[1], mac_addr[2],
25              mac_addr[3], mac_addr[4], mac_addr[5]);
26
27     Serial.print("Kirim paket ke: ");
28     Serial.print(macStr);
29     Serial.print(" | Status: ");
```

5.5.2_Praktikum_2.ino

```

31   if (sendStatus == 0) {
32       Serial.println("Berhasil Diterima");
33   } else {
34       Serial.println("Gagal (Tidak Terjangkau)");
35   }
36 }
37
38 void setup() {
39     Serial.begin(115200);
40
41     WiFi.mode(WIFI_STA);
42     WiFi.disconnect();
43
44     if (esp_now_init() != 0) {
45         Serial.println("Gagal menginisialisasi ESP-NOW!");
46         return;
47     }
48
49     esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER);
50     esp_now_register_send_cb(OnDataSent);
51
52     // Daftarkan Receiver Sela
53     esp_now_add_peer(
54         receiver1,
55         ESP_NOW_ROLE_SLAVE,
56         1,
57         NULL,
58         0
59     );

```

5.5.2_Praktikum_2.ino

```

60
61     // Daftarkan Receiver Lilis
62     esp_now_add_peer(
63         receiver2,
64         ESP_NOW_ROLE_SLAVE,
65         1,
66         NULL,
67         0
68     );
69
70     // Daftarkan Receiver Mita
71     esp_now_add_peer(
72         receiver3,
73         ESP_NOW_ROLE_SLAVE,
74         1,
75         NULL,
76         0
77     );
78
79     Serial.println("Controller Sekar ESP-NOW Siap!");
80 }
81
82 void loop() {
83     unsigned long currentMillis = millis();
84
85     if (currentMillis - previousMillis >= interval) {
86         previousMillis = currentMillis;
87

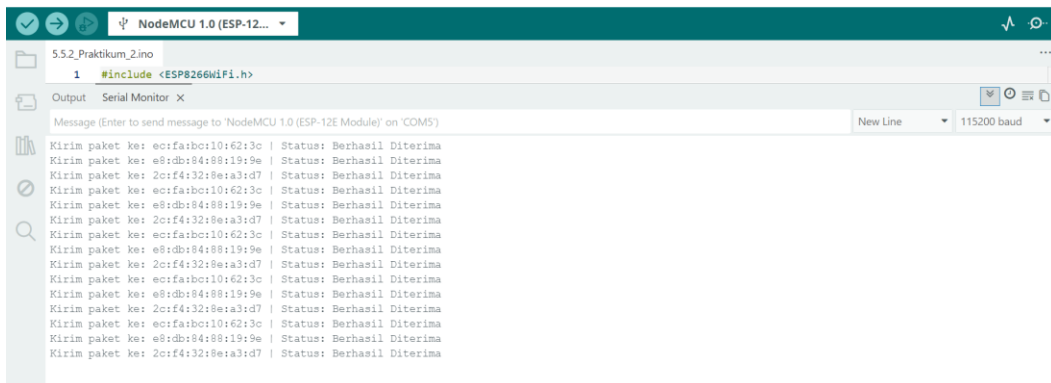
```

```

88     paketKirim.perintahId = 101;
89     paketKirim.nilaiParameter = random(10, 100);
90
91     // 0 = kirim ke semua peer yang sudah didaftarkan
92     esp_now_send(
93         0,
94         (uint8_t *) &paketKirim,
95         sizeof(paketKirim)
96     );
97 }
98 }

```

Serial monitor



Sekarang disini saya berperan sebagai penerima (slave)

Kode yang digunakan

```

5.5.2_Praktikum_2_Penerima.ino
1  #include <ESP8266WiFi.h>
2  #include <espnow.h>
3
4  // Struktur data penerima (wajib identik dengan sisi pengirim)
5  typedef struct struct_pesan {
6      int perintahId;
7      int nilaiParameter;
8  } struct_pesan;
9
10 struct_pesan paketTerima;
11
12 // Callback otomatis saat antenna mendeteksi paket masuk
13 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14     char macStr[18];
15     snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
16             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17
18     // Salin buffer memori radio langsung ke struktur data variabel
19     memcpy(&paketTerima, incomingData, sizeof(paketTerima));
20
21     Serial.print("Paket masuk dari: ");
22     Serial.println(macStr);
23     Serial.print("Ukuran Data: ");
24     Serial.print(len);
25     Serial.println(" byte");
26     Serial.print("Perintah ID: ");
27     Serial.println(paketTerima.perintahId);
28     Serial.print("Nilai Parameter: ");
29     Serial.println(paketTerima.nilaiParameter);

```


Serial monitor

5.5.2_Praktikum_2_Penerima.ino

```
1 #include <ESP8266WiFi.h>
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

```
Perintah ID: 101
Nilai Parameter: 86
-----
Paket masuk dari: ec:fa:bc:10:62:3c
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 44
-----
Paket masuk dari: ec:fa:bc:10:62:3c
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 12
-----
Paket masuk dari: ec:fa:bc:10:62:3c
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 34
-----
Paket masuk dari: ec:fa:bc:10:62:3c
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 45
-----
Paket masuk dari: ec:fa:bc:10:62:3c
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 61
-----
```

5.5.3 Praktikum 3: Pengumpulan Data Telemetri Many-to-One (ESP-NOW)

Saya disini sebagai penerima pusat (Hub Konsentrator), dan kode yang diunggah ke board adalah sebagai berikut, disini masih menggunakan kelompok yang sama seperti pada praktikum 2 dan juga hanya menggunakan perangkat esp8266:

✓ → ⚙

NodeMCU 1.0 (ESP-12... ▾)

5.5.3_Praktikum_3_PenerimaPusat.ino

```
1  #include <ESP8266WiFi.h>
2  #include <espnow.h>
3
4  typedef struct struct_sensor {
5      int id;
6      float pembacaan1;
7      float pembacaan2;
8  } struct_sensor;
9
10 struct_sensor dataTerima;
11
12 // Array penampung data terakhir: Index 0 untuk Node 1, Index 1 untuk Node 2
13 struct_sensor records[3];
14
15 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
16     memcpy(&dataTerima, incomingData, sizeof(dataTerima));
17
18     // Validasi ID agar tidak terjadi error index array out-of-bounds
19     if (dataTerima.id >= 1 && dataTerima.id <= 3) {
20         int index = dataTerima.id - 1;
21         records[index].id = dataTerima.id;
22         records[index].pembacaan1 = dataTerima.pembacaan1;
23         records[index].pembacaan2 = dataTerima.pembacaan2;
24
25         Serial.println("=====");
26         Serial.printf("DATA TERBARU DARI NODE %d\n", dataTerima.id);
27         Serial.printf("Parameter 1 : %.2f\n", records[index].pembacaan1);
28         Serial.printf("Parameter 2 : %.2f\n", records[index].pembacaan2);
29         Serial.println("=====");
```

5.5.3_Praktikum_3_PenerimaPusat.ino

```

23     records[index].pembacaan2 = dataTerima.pembacaan2;
24
25     Serial.println("=====");
26     Serial.printf("DATA TERBARU DARI NODE #%d\n", dataTerima.id);
27     Serial.printf("Parameter 1 : %.2f\n", records[index].pembacaan1);
28     Serial.printf("Parameter 2 : %.2f\n", records[index].pembacaan2);
29     Serial.println("=====");
30 }
31 }
32
33 void setup() {
34     Serial.begin(115200);
35     WiFi.mode(WIFI_STA);
36     WiFi.disconnect();
37
38     if (esp_now_init() != 0) {
39         Serial.println("Gagal Inisialisasi ESP-NOW Hub!");
40         return;
41     }
42
43     esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
44     esp_now_register_recv_cb(OnDataRecv);
45
46     Serial.println("Hub Konsentrator Siap Menerima Data Sensor...");
47 }
48
49 void loop() {
50     // Loop bebas menangani tugas lain
51 }

```

Serial Monitor

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

New Line 115200 baud

```

19:34:29.075 -> =====
19:34:29.154 -> =====
19:34:29.154 -> DATA TERBARU DARI NODE #1
19:34:29.154 -> Parameter 1 : 26.70
19:34:29.154 -> Parameter 2 : 68.10
19:34:29.154 -> =====
19:34:30.771 -> =====
19:34:30.771 -> DATA TERBARU DARI NODE #3
19:34:30.771 -> Parameter 1 : 34.20
19:34:30.771 -> Parameter 2 : 45.50
19:34:30.771 -> =====
19:34:31.715 -> =====
19:34:31.716 -> DATA TERBARU DARI NODE #2
19:34:31.716 -> Parameter 1 : 30.00
19:34:31.716 -> Parameter 2 : 50.00
19:34:31.716 -> =====
19:34:32.155 -> =====
19:34:32.155 -> DATA TERBARU DARI NODE #1
19:34:32.155 -> Parameter 1 : 25.80
19:34:32.155 -> Parameter 2 : 71.40
19:34:32.155 -> =====
19:34:33.736 -> =====
19:34:33.779 -> DATA TERBARU DARI NODE #3
19:34:33.779 -> Parameter 1 : 26.10
19:34:33.779 -> Parameter 2 : 55.30
19:34:33.779 -> =====

```

Lalu sekarang saya berperan sebagai Pengirim

Kode yang digunakan

5.5.3_Praktikum_3_Pengirim.ino

```
1  #include <ESP8266WiFi.h>
2  #include <espnow.h>
3
4  // GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)!
5  uint8_t hubMacAddress[] = {0x2C, 0xF4, 0x32, 0x8E, 0xA3, 0xD7};
6
7  // Identitas unik node (Ubah angka ini untuk node yang berbeda!)
8  #define BOARD_ID 3
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
17
18 unsigned long prevTime = 0;
19 const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik
20
21 void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
22     Serial.print("Pengiriman Node #");
23     Serial.print(BOARD_ID);
24     Serial.println(sendStatus == 0 ? " -> Sukses Diterima Hub" : " -> Gagal Sampai");
25 }
26
27 void setup() {
28     Serial.begin(115200);
29     WiFi.mode(WIFI_STA);
30     WiFi.disconnect();
31
32     if (esp_now_init() != 0) {
33         Serial.println("Inisialisasi ESP-NOW gagal");
34         return;
35     }
36
37     esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER);
38     esp_now_register_send_cb(OnDataSent);
39
40     // Daftarkan hub penerima tunggal
41     esp_now_add_peer(hubMacAddress, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);
42 }
43
44 void loop() {
45     unsigned long now = millis();
46     if (now - prevTime >= sendInterval) {
47         prevTime = now;
48
49         dataKirim.id = BOARD_ID;
50         dataKirim.pembacaan1 = random(250, 350) / 10.0; // Simulasi pembacaan suhu: 25.0 - 35.0 C
51         dataKirim.pembacaan2 = random(400, 800) / 10.0; // Simulasi pembacaan kelembapan: 40.0 - 80.0 %
52
53         esp_now_send(hubMacAddress, (uint8_t *) &dataKirim, sizeof(dataKirim));
54     }
55 }
```

Serial Monitor yang tampil

```
5.5.3_Praktikum_3_Pengirim.ino
1 #include <ESP8266WiFi.h>

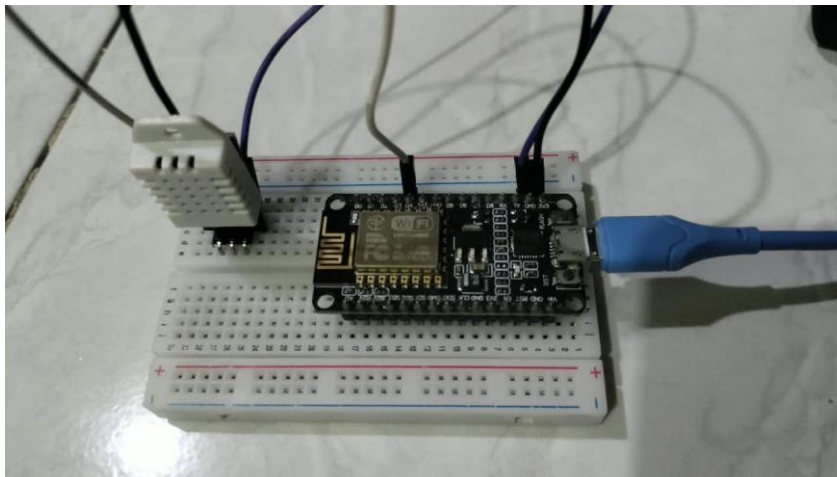
Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5') New Line 115200 baud

20:03:15.948 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:03:18.939 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:03:21.928 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:03:24.939 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:03:27.957 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:03:30.939 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:03:33.945 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:03:36.930 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:03:39.924 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:03:42.915 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:03:45.917 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:03:48.914 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:03:51.925 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:03:54.932 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:03:57.914 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:04:00.947 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:04:03.937 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:04:06.940 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:04:09.944 -> Pengiriman Node #3 -> Sukses Diterima Hub
20:04:12.950 -> Pengiriman Node #3 -> Sukses Diterima Hub
```

5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

Pada praktikum ini juga masih sama anggota kelompoknya dari yang praktikum 2, yaitu Lilis, Mita, Crosella, dan Sekar (saya sendiri)

Rangkaian fisik



Kode yang saya gunakan:

5.5.4_Praktikum_4.ino

```
1  #include <painlessMesh.h>
2  #include <DHT.h>
3  #include <ArduinoJson.h>
4
5  #define MESH_PREFIX "KosLilisMesh" // Nama jaringan Mesh bersama
6  #define MESH_PASSWORD "koslilispassword" // Kata sandi jaringan Mesh
7  #define MESH_PORT 5555 // Port komunikasi TCP Mesh
8
9  // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-3(Sekar)";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;
19
20 // Prototipe fungsi pengiriman pesan berkala
21 void sendMessage();
22 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
23
24 void sendMessage() {
25     // Pembacaan fisik data suhu dan kelembapan dari sensor DHT
26     float suhu = dht.readTemperature();
27     float kelembapan = dht.readHumidity();
28 }
```

5.5.4_Praktikum_4.ino

```

28
29 // Validasi pembacaan sensor
30 if (isnan(suhu) || isnan(kelembapan)) {
31     Serial.println("[DHT Error] Gagal membaca data dari sensor DHT!");
32     return;
33 }
34
35 // Rangkai data telemetri ke dalam format JSON menggunakan ArduinoJson
36 StaticJsonDocument<200> doc;
37 doc["node"] = nodeName;
38 doc["chipId"] = mesh.getNodeId();
39 doc["suhu"] = suhu;
40 doc["kelembapan"] = kelembapan;
41
42 String msg;
43 serializeJson(doc, msg);
44
45 // Siarkan paket JSON ke seluruh jaringan mesh (multi-hop broadcast)
46 mesh.sendBroadcast(msg);
47
48 Serial.print("[KIRIM MESH] ");
49 Serial.println(msg);
50 }
51
52 // Callback otomatis saat menerima pesan dari node manapun di jaringan mesh
53 void receivedCallback(uint32_t from, String &msg) {
54     // Parsing muatan teks JSON yang diterima
55     StaticJsonDocument<200> doc;
56     DeserializationError error = deserializeJson(doc, msg);

```

5.5.4_Praktikum_4.ino

```

57
58 if (!error) {
59     const char* sender = doc["node"];
60     uint32_t chipId = doc["chipId"];
61     float suhu = doc["suhu"];
62     float kelembapan = doc["kelembapan"];
63
64     Serial.println("=====");
65     Serial.printf("[TERIMA DARI] %s (Node ID: %u | Chip ID: %u)\n", sender, from, chipId);
66     Serial.printf("Suhu : %.2f °C\n", suhu);
67     Serial.printf("Kelembapan : %.2f %%\n", kelembapan);
68     Serial.println("=====");
69 } else {
70     Serial.printf("[TERIMA DATA MENTAH DARI %u]: %s\n", from, msg.c_str());
71 }
72 }
73
74 // Callback otomatis saat ada perangkat baru bergabung ke mesh
75 void newConnectionCallback(uint32_t nodeId) {
76     Serial.printf("--> Koneksi Baru Terdeteksi! Node ID: %u\n", nodeId);
77 }
78
79 // Callback otomatis saat topologi rantai mesh berubah (node keluar/masuk)
80 void changedConnectionCallback() {
81     Serial.println("--> Topologi rantai mesh telah diperbarui");
82 }
83

```

```

83
84 void setup() {
85     Serial.begin(115200);
86
87     // Inisialisasi sensor DHT
88     dht.begin();
89
90     // Atur level log debugging (hanya tampilkan error dan status startup)
91     mesh.setDebugMsgTypes(ERROR | STARTUP);
92
93     // Inisialisasi jaringan mesh
94     mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);
95
96     // Daftarkan event callback
97     mesh.onReceive(&receivedCallback);
98     mesh.onNewConnection(&newConnectionCallback);
99     mesh.onChangedConnections(&changedConnectionCallback);
100
101     // Jadwalkan tugas pengiriman data berkala menggunakan TaskScheduler
102     userScheduler.addTask(taskSendMessage);
103     taskSendMessage.enable();
104
105     Serial.printf("Mesh Node [%s] Berjalan. Menunggu pembentukan topologi...\n", nodeName);
106 }
107
108 void loop() {
109     // Wajib dipanggil rutin untuk memproses perutean paket dan sinkronisasi waktu
110     mesh.update();
111 }

```

Serial Monitor

5.5.4_Praktikum_4.ino

```
1 #include <painlessMesh.h>
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

```

20:31:18.977 -> =====
20:31:19.308 -> =====
20:31:19.308 -> [TERIMA DARI] Node-2(sella) (Node ID: 3155190332 | Chip ID: 3155190332)
20:31:19.346 -> Suhu      : 1.70 °C
20:31:19.346 -> Kelembapan : 21.00 %
20:31:19.346 -> =====
20:31:19.793 -> [KIRIM MESH] {"node":"Node-3(Sekar)","chipId":974571082,"suhu":32.7,"kelembapan":77.2}
20:31:20.896 -> =====
20:31:20.896 -> [TERIMA DARI] Node-4(Mita) (Node ID: 848208855 | Chip ID: 848208855)
20:31:20.896 -> Suhu      : 32.20 °C
20:31:20.896 -> Kelembapan : 68.70 %
20:31:20.896 -> =====
20:31:21.931 -> =====
20:31:21.931 -> [TERIMA DARI] Node-1(lilis) (Node ID: 2223511966 | Chip ID: 2223511966)
20:31:21.965 -> Suhu      : 32.30 °C
20:31:21.965 -> Kelembapan : 69.40 %
20:31:21.965 -> =====
20:31:22.371 -> =====
20:31:22.371 -> [TERIMA DARI] Node-2(sella) (Node ID: 3155190332 | Chip ID: 3155190332)
20:31:22.371 -> Suhu      : 1.70 °C
20:31:22.371 -> Kelembapan : 21.10 %
20:31:22.371 -> =====
20:31:22.790 -> [KIRIM MESH] {"node":"Node-3(Sekar)","chipId":974571082,"suhu":32.7,"kelembapan":78.3}
20:31:23.856 -> =====
20:31:23.856 -> [TERIMA DARI] Node-4(Mita) (Node ID: 848208855 | Chip ID: 848208855)
20:31:23.896 -> Suhu      : 32.20 °C
20:31:23.896 -> Kelembapan : 68.50 %
20:31:23.896 -> =====

```


5.6 Latihan

- **Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):** Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:
 - i. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
 - ii. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?
 - iii. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

Jawab:

```
Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')
20:51:33.527 -> Kirim paket ke: ec:fa:bc:10:62:3c | Status: Berhasil Diterima
20:51:33.527 -> Kirim paket ke: e8:db:84:88:19:9e | Status: Berhasil Diterima
20:51:33.527 -> Kirim paket ke: 2c:f4:32:8e:a3:d7 | Status: Berhasil Diterima
20:51:35.526 -> Kirim paket ke: ec:fa:bc:10:62:3c | Status: Berhasil Diterima
20:51:35.526 -> Kirim paket ke: e8:db:84:88:19:9e | Status: Berhasil Diterima
20:51:35.526 -> Kirim paket ke: 2c:f4:32:8e:a3:d7 | Status: Berhasil Diterima
20:51:37.528 -> Kirim paket ke: ec:fa:bc:10:62:3c | Status: Berhasil Diterima
20:51:37.528 -> Kirim paket ke: e8:db:84:88:19:9e | Status: Berhasil Diterima
20:51:37.528 -> Kirim paket ke: 2c:f4:32:8e:a3:d7 | Status: Berhasil Diterima
20:51:39.490 -> Kirim paket ke: ec:fa:bc:10:62:3c | Status: Berhasil Diterima
20:51:39.525 -> Kirim paket ke: e8:db:84:88:19:9e | Status: Berhasil Diterima
20:51:39.525 -> Kirim paket ke: 2c:f4:32:8e:a3:d7 | Status: Berhasil Diterima
20:51:41.500 -> Kirim paket ke: ec:fa:bc:10:62:3c | Status: Berhasil Diterima
20:51:41.500 -> Kirim paket ke: e8:db:84:88:19:9e | Status: Berhasil Diterima
20:51:41.536 -> Kirim paket ke: 2c:f4:32:8e:a3:d7 | Status: Gagal (Tidak Terjangkau)
20:51:43.491 -> Kirim paket ke: ec:fa:bc:10:62:3c | Status: Berhasil Diterima
20:51:43.536 -> Kirim paket ke: e8:db:84:88:19:9e | Status: Berhasil Diterima
20:51:43.536 -> Kirim paket ke: 2c:f4:32:8e:a3:d7 | Status: Gagal (Tidak Terjangkau)
```

Saat salah satu board penerima dicabut, status pengiriman pada Serial Monitor Controller berubah dari “Berhasil Diterima” menjadi “Gagal (Tidak Terjangkau)” pada MAC Address board tersebut. Hal ini terjadi karena ESP-NOW bekerja pada Layer 2 MAC dan menggunakan MAC Address sebagai tujuan. Saat paket berhasil diterima, perangkat penerima memberikan ACK sehingga `sendStatus == 0`. Jika board dimatikan, ACK tidak diterima sehingga Controller mendeteksi pengiriman gagal. Jadi,

walaupun tidak menggunakan TCP, ESP-NOW tetap bisa mengetahui apakah paket berhasil diterima atau tidak.

- **Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):** Jalankan jaringan painlessMesh pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID:

....

- i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?
- ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

Jawab:

Setelah salah satu node di-reset atau dimatikan, `changedConnectionCallback()` mendeteksi perubahan topologi sekitar 5 detik. Hal ini menunjukkan bahwa jaringan mesh dapat mendeteksi ketika salah satu node keluar dari jaringan.

Saat node dinyalakan kembali, node tersebut membutuhkan sekitar 10 detik untuk terhubung kembali secara otomatis tanpa konfigurasi ulang. Hal ini menunjukkan sifat self-healing pada painlessMesh, yaitu jaringan dapat memperbarui dan membentuk kembali koneksinya secara otomatis.

```
Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

20:59:29.079 -> Suhu : 31.90 °C
20:59:29.079 -> Kelembapan : 69.70 %
20:59:29.079 -> =====
20:59:29.111 -> [KIRIM MESH] {"node":"Node-3 (Sekar) ", "chipId":974571082, "suhu":31.6, "kelembapan":69.6}
20:59:32.048 -> =====
20:59:32.093 -> [TERIMA DARI] Node-1 (lilis) (Node ID: 2223511966 | Chip ID: 2223511966)
20:59:32.093 -> Suhu : 31.90 °C
20:59:32.093 -> Kelembapan : 69.70 %
20:59:32.093 -> =====
20:59:32.125 -> [KIRIM MESH] {"node":"Node-3 (Sekar) ", "chipId":974571082, "suhu":31.6, "kelembapan":69.8}
20:59:35.063 -> =====
20:59:35.063 -> [TERIMA DARI] Node-1 (lilis) (Node ID: 2223511966 | Chip ID: 2223511966)
20:59:35.063 -> Suhu : 31.90 °C
20:59:35.063 -> Kelembapan : 69.60 %
20:59:35.063 -> =====
20:59:35.107 -> [KIRIM MESH] {"node":"Node-3 (Sekar) ", "chipId":974571082, "suhu":31.6, "kelembapan":69.8}
20:59:38.035 -> =====
20:59:38.035 -> [TERIMA DARI] Node-1 (lilis) (Node ID: 2223511966 | Chip ID: 2223511966)
20:59:38.035 -> Suhu : 31.90 °C
20:59:38.076 -> Kelembapan : 69.60 %
20:59:38.076 -> =====
20:59:38.108 -> [KIRIM MESH] {"node":"Node-3 (Sekar) ", "chipId":974571082, "suhu":31.6, "kelembapan":69.9}
20:59:41.055 -> =====
20:59:41.091 -> [TERIMA DARI] Node-1 (lilis) (Node ID: 2223511966 | Chip ID: 2223511966)
20:59:41.091 -> Suhu : 31.90 °C
20:59:41.091 -> Kelembapan : 69.60 %
20:59:41.091 -> =====
20:59:41.127 -> [KIRIM MESH] {"node":"Node-3 (Sekar) ", "chipId":974571082, "suhu":31.6, "kelembapan":69.8}

Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

21:04:34.697 -> [KIRIM MESH] {"node":"Node-3 (Sekar) ", "chipId":974571082, "suhu":32.1, "kelembapan":68.8}
21:04:37.697 -> [KIRIM MESH] {"node":"Node-3 (Sekar) ", "chipId":974571082, "suhu":32.1, "kelembapan":68.3}
21:04:40.715 -> [KIRIM MESH] {"node":"Node-3 (Sekar) ", "chipId":974571082, "suhu":32.1, "kelembapan":68.6}
21:04:43.697 -> [KIRIM MESH] {"node":"Node-3 (Sekar) ", "chipId":974571082, "suhu":32.1, "kelembapan":68.6}
21:04:47.287 -> [KIRIM MESH] {"node":"Node-3 (Sekar) ", "chipId":974571082, "suhu":32.1, "kelembapan":68.3}
21:04:47.581 -> --> Koneksi Baru Terdeteksi! Node ID: 848208855
21:04:47.581 -> --> Topologi rantai mesh telah diperbarui
21:04:48.195 -> =====
21:04:48.195 -> [TERIMA DARI] Node-2 (sella) (Node ID: 3155190332 | Chip ID: 3155190332)
21:04:48.195 -> Suhu : 1.40 °C
21:04:48.195 -> Kelembapan : 20.40 %
21:04:48.195 -> =====
21:04:49.420 -> =====
21:04:49.420 -> [TERIMA DARI] Node-4 (Mita) (Node ID: 848208855 | Chip ID: 848208855)
21:04:49.420 -> Suhu : 32.00 °C
21:04:49.420 -> Kelembapan : 67.20 %
21:04:49.420 -> =====
21:04:49.701 -> [KIRIM MESH] {"node":"Node-3 (Sekar) ", "chipId":974571082, "suhu":32.1, "kelembapan":68.6}
21:04:50.038 -> =====
21:04:50.077 -> [TERIMA DARI] Node-1 (lilis) (Node ID: 2223511966 | Chip ID: 2223511966)
21:04:50.077 -> Suhu : 31.90 °C
21:04:50.077 -> Kelembapan : 69.80 %
21:04:50.077 -> =====
21:04:51.180 -> =====
21:04:51.216 -> [TERIMA DARI] Node-2 (sella) (Node ID: 3155190332 | Chip ID: 3155190332)
21:04:51.216 -> Suhu : 1.40 °C
21:04:51.216 -> Kelembapan : 20.40 %
21:04:51.216 -> =====
```

- **Latihan 3 (Eksperimen Modifikasi Filter Telemetri & Ambang Batas JSON - Praktikum 4):** Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetri dari semua pengirim tanpa penyaringan.
 - i. Modifikasilah fungsi receivedCallback pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetri penuh ke Serial Monitor jika suhu

yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).

- ii. Jika suhu bernilai normal (≤ 32.0 °C), program cukup mencetak status singkat satu baris: [NORMAL] Telemetry dari Node <chipId> aman.
- iii. Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!

Jawab:

Blok fungsi modifikasi

latihan_3.ino

```
52 // Callback otomatis saat menerima pesan dari node manapun di jaringan mesh (Latihan 3 Filter Telemetry)
53 void receivedCallback(uint32_t from, String &msg) {
54     // Parsing muatan teks JSON yang diterima
55     StaticJsonDocument<200> doc;
56     DeserializationError error = deserializeJson(doc, msg);
57
58     if (!error) {
59         const char* sender = doc["node"];
60         uint32_t chipId = doc["chipId"];
61         float suhu = doc["suhu"];
62         float kelembapan = doc["kelembapan"];
63
64         // Filter Ambang Batas Suhu Kritis (> 32.0 °C)
65         if (suhu > 32.0) {
66             Serial.println("=====");
67             Serial.println("          [PERINGATAN SUHU KRITIS]          ");
68             Serial.println("=====");
69             Serial.printf("[TERIMA DARI] %s (Node ID: %u | Chip ID: %u)\n", sender, from, chipId);
70             Serial.printf("Suhu : %.2f °C\n", suhu);
71             Serial.printf("Kelembapan : %.2f %%\n", kelembapan);
72             Serial.println("=====");
73         } else {
74             // Format ringkas jika suhu normal (<= 32.0 °C)
75             Serial.printf("[NORMAL] Telemetry dari Node %u aman.\n", chipId);
76         }
77     } else {
78         Serial.printf("[TERIMA DATA MENTAH DARI %u]: %s\n", from, msg.c_str());
79     }
80 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

```
21:18:29.750 -> [NORMAL] Telemetry dari Node 3155190332 aman.
21:18:30.431 -> [NORMAL] Telemetry dari Node 2223511966 aman.
21:18:30.632 -> [NORMAL] Telemetry dari Node 2223511966 aman.
21:18:31.375 -> [KIRIM MESH] {"node":"Node-3(Sekar)", "chipId":974571082, "suhu":32.2, "kelembapan":69.5}
21:18:32.423 -> [NORMAL] Telemetry dari Node 848208855 aman.
21:18:33.416 -> [NORMAL] Telemetry dari Node 3155190332 aman.
21:18:33.617 -> [NORMAL] Telemetry dari Node 2223511966 aman.
21:18:34.350 -> [KIRIM MESH] {"node":"Node-3(Sekar)", "chipId":974571082, "suhu":32.2, "kelembapan":69.2}
21:18:35.398 -> [NORMAL] Telemetry dari Node 848208855 aman.
21:18:36.500 -> [NORMAL] Telemetry dari Node 3155190332 aman.
21:18:36.765 -> [NORMAL] Telemetry dari Node 2223511966 aman.
21:18:37.350 -> [KIRIM MESH] {"node":"Node-3(Sekar)", "chipId":974571082, "suhu":32.2, "kelembapan":69}
21:18:38.375 -> [NORMAL] Telemetry dari Node 848208855 aman.
21:18:39.429 -> [NORMAL] Telemetry dari Node 3155190332 aman.
21:18:39.583 -> [NORMAL] Telemetry dari Node 2223511966 aman.
21:18:40.375 -> [KIRIM MESH] {"node":"Node-3(Sekar)", "chipId":974571082, "suhu":32.2, "kelembapan":69}
21:18:41.415 -> [NORMAL] Telemetry dari Node 848208855 aman.
21:18:42.416 -> [NORMAL] Telemetry dari Node 3155190332 aman.
21:18:42.531 -> [NORMAL] Telemetry dari Node 2223511966 aman.
```